

Abstract

- WarehouseIQ is a web-based warehouse and inventory management system for managing products, suppliers, warehouses and stock movements. It provides inventory monitoring, stock-in/stock-out operations and warehouse information through a React, Flask REST API and MySQL-based system.
- Outcome: Centralized and efficient inventory tracking with improved visibility of stock operations.

Problem Statement & Objectives

- Problem:** Traditional/manual inventory management can make it difficult to maintain accurate stock information, track stock movements and manage multiple warehouses efficiently.
- Objectives:** Manage products and warehouses, Track inventory levels, Record stock IN/OUT, Centralize stock information.
- Scope:** Users, Products, Inventory, Warehouses, Suppliers, Stock Movements.

Existing vs Proposed System

Feature	Existing System	Proposed System
Inventory Tracking	Manual / fragmented	Centralized
Stock Movement	Manual records	Digital tracking
Warehouse Data	Separate records	Integrated database
Supplier Data	Difficult to manage	Centralized management
[Limitation / Gap]	Errors & limited visibility	Automated and accessible system

Feasibility Study

- Technical:** React, Flask, MySQL and Docker provide the required development and deployment environment.
- Economic:** Uses open-source technologies with low software cost.
- Operational:** Provides simple interfaces for inventory and warehouse operations.
- Schedule:** Modules can be developed and integrated within the project timeline.

Results & Testing

- User login and authentication tested.
- Product and warehouse data retrieved successfully.
- Inventory levels displayed from MySQL.
- Stock-in and stock-out operations tested.
- Stock movements recorded successfully.
- REST API and database connectivity verified.

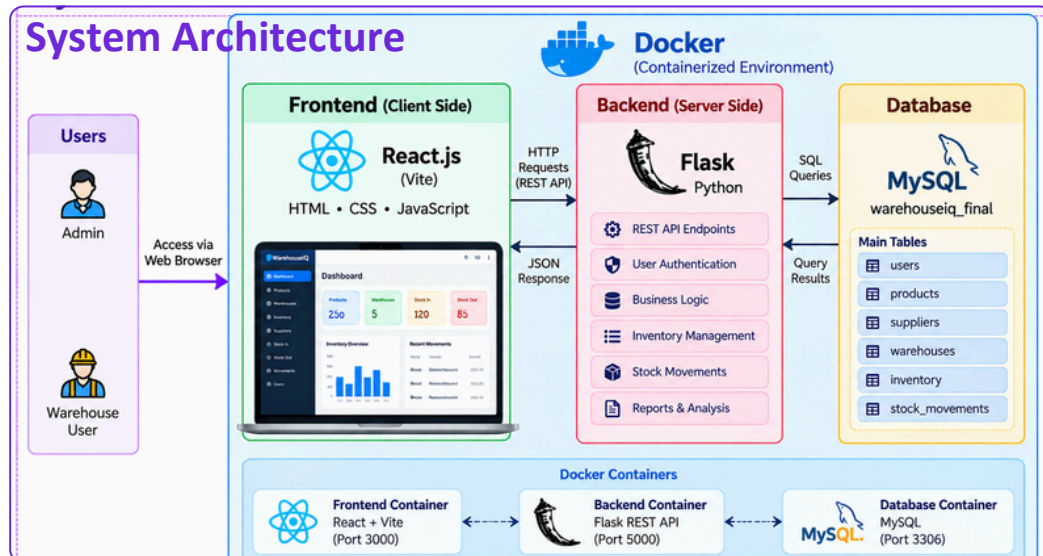
Conclusion

WarehouseIQ successfully integrates warehouse, inventory, product and supplier management into a single database-driven system. The system provides centralized inventory tracking and supports stock-in, stock-out and stock movement operations.

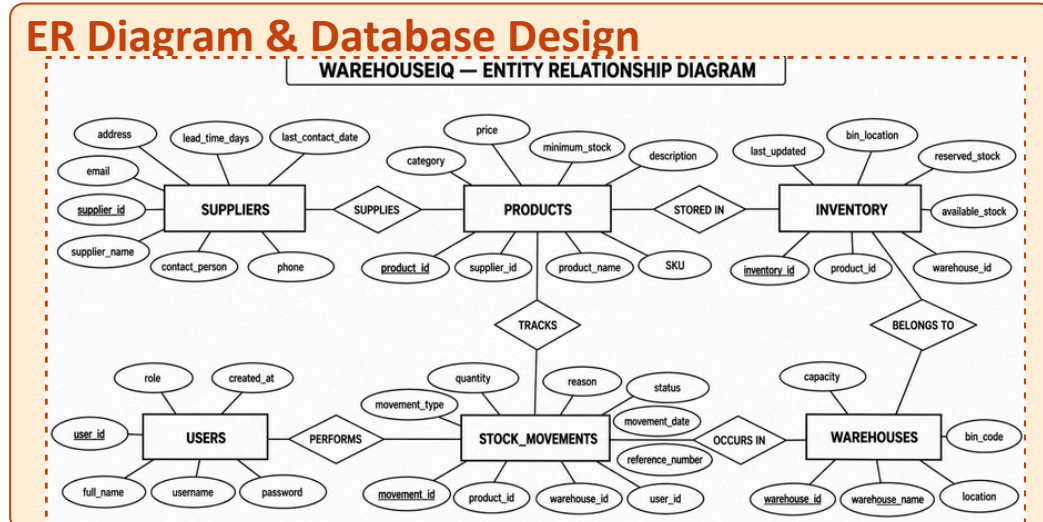
Tools & Technologies

- Front-End: React.js, Vite, HTML, CSS
- Back-End: Python, Flask, REST APIs
- Database: MySQL
- Containerization: Docker
- Tools: Git, GitHub, VS Code, Postman, Draw.io

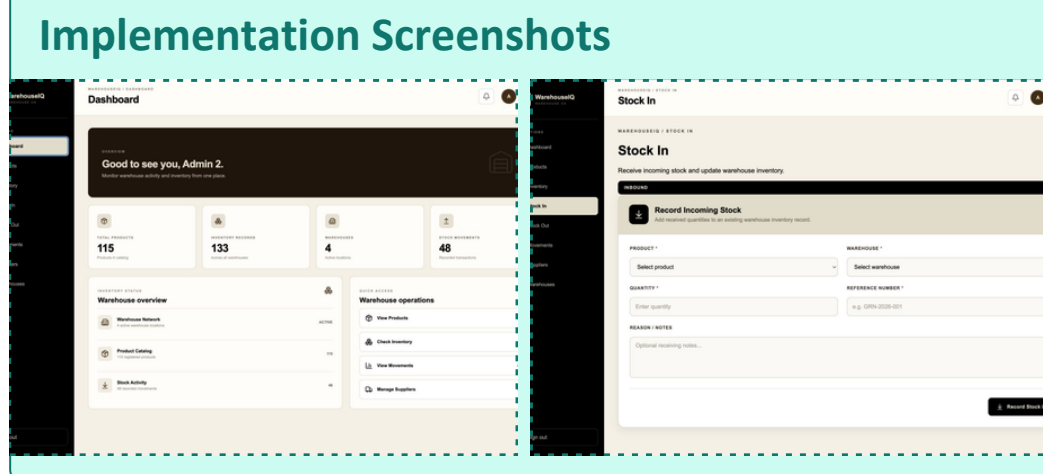
System Architecture



ER Diagram & Database Design



Implementation Screenshots



Future Scope

- Barcode/QR code-based inventory tracking.
- Automated low-stock notifications.
- Advanced inventory reports and analytics.
- Role-based access control and improved security.

